

Problem 01: Find waiting time, turn-around time for each process and average waiting time
Using Priority Scheduling algorithm.

Source Code:

```
#pragma GCC optimize("Ofast")
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct tpl{
    int p, bt, prio;
};

void Solve()
{
    int n;
    cin >> n;
    cout << "Total Process: " << n << "\n";
    vector<tpl> proc(n);
    for(int i=0; i<n; i++){
        cin >> proc[i].bt >> proc[i].prio;
        proc[i].p = i+1;
    }

    sort(proc.begin(), proc.end(), [&](auto &a, auto &b){
        return a.prio < b.prio;
```

```
});
```

```
vector<int> tat(n), wt(n);
```

```
int time = 0;
```

```
double avgwt = 0;
```

```
for(int i=0; i<n; i++){
```

```
    time += proc[i].bt;
```

```
    tat[i] = time;
```

```
    wt[i] = tat[i] - proc[i].bt;
```

```
    avgwt += wt[i];
```

```
}
```

```
cout << "P BT PRIO TAT   WT\n";
```

```
for(int i=0; i<n; i++){
```

```
    cout << proc[i].p << " " << proc[i].bt << " " << proc[i].prio << "   "
```

```
        << tat[i] << "   " << wt[i] << "\n";
```

```
}
```

```
cout << fixed << setprecision(2);
```

```
cout << "Average Waiting Time: " << avgwt / n << "\n";
```

```
}
```

```
int32_t main()
```

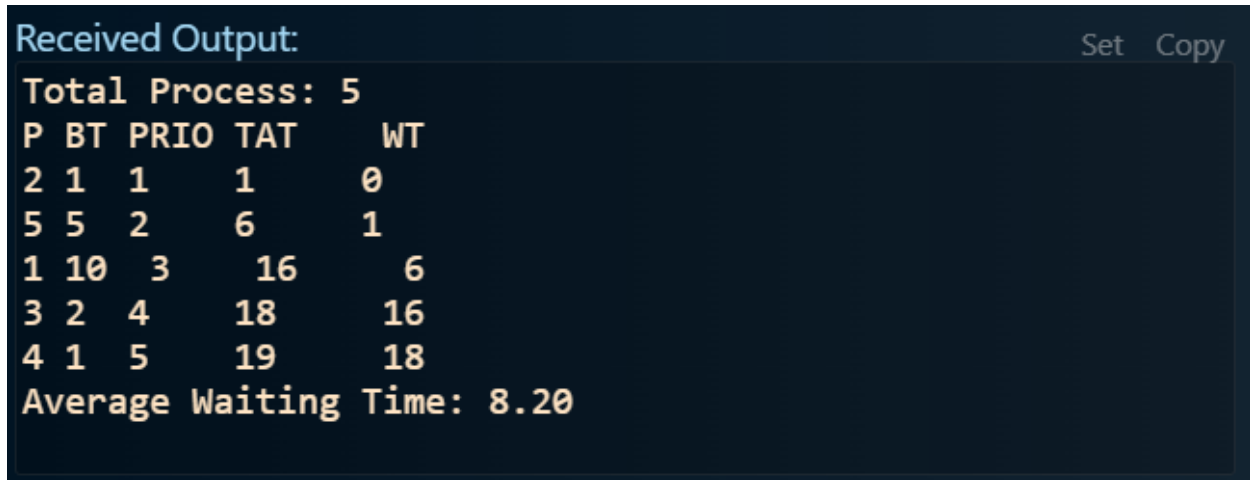
```
{
```

```
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
```

```
    ll t = 1;
```

```
// cin >> t;  
while (t--)  
    Solve();  
return 0;  
}
```

Output:

A terminal window with a dark blue background and light green text. The title bar reads 'Received Output:'. In the top right corner, there are two buttons labeled 'Set' and 'Copy'. The output text is as follows:

```
Total Process: 5  
P BT PRIO TAT WT  
2 1 1 1 0  
5 5 2 6 1  
1 10 3 16 6  
3 2 4 18 16  
4 1 5 19 18  
Average Waiting Time: 8.20
```

P	BT	PRIO	TAT	WT
2	1	1	1	0
5	5	2	6	1
1	10	3	16	6
3	2	4	18	16
4	1	5	19	18

Figure 1: Output of problem 01.

Problem 02: Find waiting time, turn-around time for each process and average waiting time
Using Round Robin algorithm.

Source Code:

```
#pragma GCC optimize("Ofast")
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct tpl {
    int p, bt;
};

void Solve()
{
    int n, q;
    cin >> n >> q;
    cout << "Total Process: " << n << "\n";
    vector<tpl> proc(n);
    int pid, bt;
    for(int i=0; i<n; i++){
        cin >> pid >> bt;
        proc[i].p = pid;
        proc[i].bt = bt;
    }
```

```
vector<int> rem(n), tat(n), wt(n);  
for(int i=0; i<n; i++) rem[i] = proc[i].bt;
```

```
int time = 0;
```

```
bool done = false;
```

```
while(!done){  
    done = true;  
    for(int i=0; i<n; i++){  
        if(rem[i] > 0){  
            done = false;  
            if(rem[i] > q){  
                time += q;  
                rem[i] -= q;  
            } else {  
                time += rem[i];  
                tat[i] = time;  
                rem[i] = 0;  
            }  
        }  
    }  
}
```

```
double avgwt = 0;
```

```
for(int i=0; i<n; i++){  
    wt[i] = tat[i] - proc[i].bt;  
    avgwt += wt[i];  
}
```

```

    }

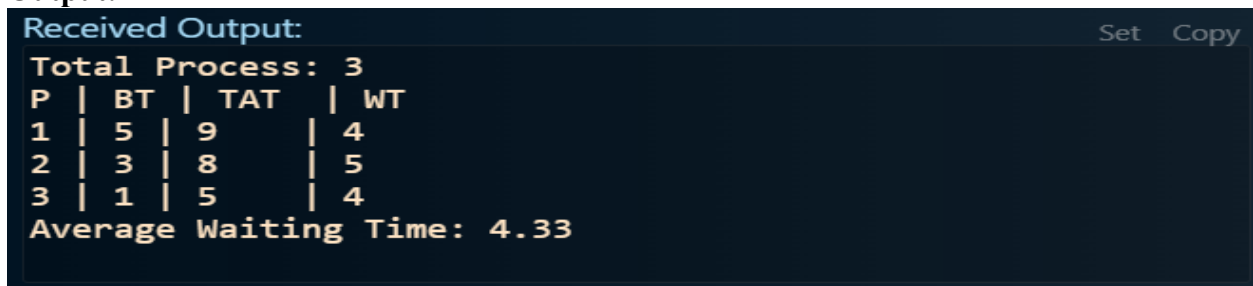
    cout << "P | BT | TAT | WT\n";
    for(int i=0; i<n; i++){
        cout << proc[i].p << " | " << proc[i].bt << " | " << tat[i] << " | " << wt[i] << "\n";
    }

    cout << fixed << setprecision(2);
    cout << "Average Waiting Time: " << avgwt / n << "\n";
}

int32_t main()
{
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    ll t = 1;
    // cin >> t;
    while (t--)
        Solve();
    return 0;
}

```

Output:



```

Received Output:
Total Process: 3
P | BT | TAT | WT
1 | 5 | 9 | 4
2 | 3 | 8 | 5
3 | 1 | 5 | 4
Average Waiting Time: 4.33

```

Figure 2: Output of problem 02.